Destination costs of metrics

One of the most challenging aspects of metrics is what we call *cardinality explosion*, which we will discuss in detail in chapter 5. What does this mean on a high level? Well, in many cases, the dimension of a metric only has a small set of possible values. For example, the number of HTTP status codes (https://httpstatuses.com/) may be limited to a couple dozen:

```
myapp_http_requests_total{code="200", handler="/helloworld"} --> 42
myapp_http_requests_total{code="404", handler="/cryptomining"} --> 5
...
```

This doesn't cause any issues, as the time series databases are well able to handle it, from a storage and query perspective. But what if you decide to encode the user ID into a label as follows?

```
myapp_http_requests_total{code="200", handler="/ex1", uid="9876" } --> 567
```

Congratulations! You have just introduced the potential for cardinality explosion. Why? Well, imaging your service becomes popular and you end up having to deal with tens or hundreds of thousands—or maybe even millions— of users.

There can be as many values for the `uid` label as you have users, which is very expensive for the time series database to deal with, both in terms of storage and query performance, if possible at all (usually, you will see your queries getting slower in such a case). There's really only one solution: keep your dimensions in control.

Another source of costs in the destination comes when there are many sources. Say you have a fleet of Kubernetes clusters and want to centrally query the metrics. You would need to consider how you go about long-term storage and federation (http://mng.bz/6Ddo) of your metrics, something we will get back to in more detail in chapter 5.

Benefits of metrics

By the very nature of metrics, their numeric values enable all sorts of automation. For example, you don't have to manually track a metric by looking at a dashboard, but you define a condition (e.g., "goes above this value for at least 3 minutes") and get a notification if and when this is the case. That kind of automation is usually called *alerting*.

Then, there's the wide area of establishing a baseline. (How does the metric "normally" look—what range, max, and min does it have?) One can employ fancy methods here as well, and there's a whole area, called *anomaly detection*, dedicated to the topic of why something looks abnormal. Again, we return to this topic in greater detail later in the book.